

C++ Dependency Management: Package Consumption vs Development

Document: [P1067R0](#)
Audience: SG15
Authors: Boris Kolpackov
Reply-To: boris@codesynthesis.com
Date: 2018-05-04

Abstract

Today's C++ package managers focus on package consumption leaving the question of what happens during the package development largely unanswered. In this presentation I would like to demonstrate the dependency management workflow focused on development (as opposed to consumption) that we have implemented in the [build2](#) toolchain.

Background

Current C++ package dependency managers provide roughly the same functionality: there is a repository of *released* packages that may depend on each other and which the user can retrieve and install with the package manager making sure all the dependencies are satisfied. In other words, these package managers are *consumption* tools with the *dependency management* functionality boiling down to satisfying dependencies of the packages being consumed.

The question is then what happens between the releases of these packages? Or, in other words, how are they developed, tested, and delivered? Currently, the answer

is *painfully* (and rarely by mere mortals).

At the same time, languages competing with C++ show that the state of the art has moved on (see Rust's `cargo`, Go's `dep` [design paper](#) and, more recently, `vgo`). As their experience demonstrate, a package manager alone is not sufficient: we also need support for managing dependencies of a project during development – a tool often referred to as the *project dependency manager*.

And the *dependency management* functionality in this context is quite a bit different: we are talking about adding and removing dependencies (to/from the project's `manifest` and propagating the result to its build configurations) as well as managing their upgrades and downgrades all while making sure the precise set of dependencies can be reproduced by everyone else (by storing their exact versions in the project's `lockfile`).

It is not hard to see that a project dependency manager can be naturally built on top of the package dependency manager. In fact, the languages mentioned above do not separate the two (and also combine the dependency manager with the build system, which has its drawbacks). There are, however, a number of additional requirements and challenges that will be hard to address without a well-integrated (note: *integrated*, not *combined*) C++ build toolchain:

- Support for automated, continuous versioning between releases.
- Dependence on unpackaged/unreleased revisions in version control repositories.
- Dependency auto-synchronization on build system invocations.